

[Assignment V]

Problem 1: Implementing backprop for MLP. Consider the multi-layer perceptron

$$\begin{aligned}
 y_L &= A_L y_{L-1} + b_L \\
 y_{L-1} &= \sigma(A_{L-1} y_{L-2} + b_{L-1}) \\
 &\vdots \\
 y_2 &= \sigma(A_2 y_1 + b_2) \\
 y_1 &= \sigma(A_1 x + b_1),
 \end{aligned}$$

(4층 MLP)

where $x \in \mathbb{R}^{n_0}$, $A_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $b_\ell \in \mathbb{R}^{n_\ell}$, and $n_L = 1$. Let $\sigma(z) = (1 + e^{-z})^{-1}$ be the sigmoid activation function. Let $f_\theta(x) = y_L$ and consider the loss function

$$\ell(\theta) = \frac{1}{2} (f_\theta(X_{\text{data}}) - Y_{\text{data}})^2.$$

Download the starter code `mlp_backprop.py` and implement backprop using the gradient computation of homework 4 problem 6. Your code should roughly be of the form:

```

A_list, b_list = [], []
for _ in range(L-1):
    A_list.append(torch.rand(4, 4))
    b_list.append(torch.rand(4, 1))
A_list.append(torch.rand(1, 4))
b_list.append(torch.rand(1, 1))

```

$A_1, \dots, A_{L-1}, b_1, \dots, b_{L-1}$ 은 4×4 , 4×1 size의 텐서

A_L, b_L 은 1×4 , 1×1 size의 텐서

```

# Option 3: implement backprop yourself
y_list = [X_data]
y = X_data
for ell in range(L):
    S = sigma if ell < L-1 else lambda x: x
    y = S(A_list[ell]@y+b_list[ell])
    y_list.append(y)

```

$y_1, y_2, \dots, y_L$ 은 sigmoid를 거쳐

(↓ Back-propagation)

```

dA_list = []
db_list = []
dy = y - Y_data # dloss/dy_L
for ell in reversed(range(L)):
    S = sigma_prime if ell < L-1 else lambda x: torch.ones(x.shape)
    A, b, y = A_list[ell], b_list[ell], y_list[ell]
    v = S(torch.reshape(A@y+b, (-1,)))
    db = dy@torch.diag(v) # dloss/db_ell
    dA = torch.diag(v)@torch.t(dy)@torch.t(y) # dloss/dA_ell
    dy = dy@torch.diag(v)@A # dloss/dy_{ell-1}, for the next iteration
    dA_list.insert(0, dA)
    db_list.insert(0, db)

print(dA_list[0])

```

→ y_{list} 은 0인 X_{data} 이므로
인덱스 0인 A, b 는
append를 했으므로 인덱스 0인 y
→ vector v 를 $A@y+b$ 로 계산

$\ell = [0, 1, 2, 3]$ 의 경우 y 는 1이 다음, 0이 3번 반복.

$$\begin{aligned}
 \frac{\partial \ell}{\partial b_i} &= \frac{\partial \ell}{\partial y_i} \frac{\partial y_i}{\partial b_i} = dy \cdot \text{diag}(\sigma'(A_{L-1} y_{L-2} + b_{L-1})) \\
 &= dy \cdot \text{diag}(\sigma'(A_{L-1} y + b))
 \end{aligned}$$

y_L 의 $\partial \ell / \partial y_L$ 1이
4개의 y_L , σ 의 $\partial \ell / \partial y_L$
0이 23번 반복

$$\begin{aligned}
 \frac{\partial \ell}{\partial y_i} &= \frac{\partial \ell}{\partial y_L} \frac{\partial y_L}{\partial y_i} = \frac{\partial \ell}{\partial y_L} \text{diag}(\sigma'(A_{L-1} y + b)) \left(\frac{\partial y_L}{\partial y_i} \right)^T y_{L-1}^T \\
 &= \text{diag}(\sigma'(z)) \left(\frac{\partial \ell}{\partial y_i} \right)^T y_{L-1}^T \\
 &= \text{diag}(\sigma'(z)) \left(\frac{\partial \ell}{\partial y_i} \right)^T
 \end{aligned}$$

$$\frac{\partial \ell}{\partial y_i} = \frac{\partial \ell}{\partial y_{L-1}} \frac{\partial y_{L-1}}{\partial y_i} = \frac{\partial \ell}{\partial y_{L-1}} \text{diag}(\sigma'(z)) A_i$$

Problem 2: Vanishing gradients. When training very deep neural networks, one often encounters the problem of vanishing gradients. Consider the MLP of homework 4 problem 6. Assume the activation function σ is the sigmoid activation function. Define $\tilde{y}_i = A_i y_{i-1} + b_i$ for $i = 1, \dots, L$. So $y_L = \tilde{y}_L$ and $y_i = \sigma(\tilde{y}_i)$ for $i = 1, \dots, L-1$.

Assume the matrices $A_1, \dots, A_L$ are all not too large. If A_j is small for some $j \in \{\ell+1, \dots, L\}$, then

$$\frac{\partial y_L}{\partial b_i}, \quad \frac{\partial y_L}{\partial A_i}$$

for $i = 1, \dots, \ell$ become small. If $\tilde{y}_j$ has large absolute value for some $j \in \{\ell+1, \dots, L-1\}$, then

$$\frac{\partial y_L}{\partial b_i}, \quad \frac{\partial y_L}{\partial A_i}$$

for $i = 1, \dots, \ell$ become small. Explain why this is the case.

(1) $\frac{\partial y_L}{\partial b_i} = \frac{\partial y_L}{\partial y_{L-1}} \frac{\partial y_{L-1}}{\partial y_{L-2}} \dots \frac{\partial y_{L-1}}{\partial y_i} \frac{\partial y_i}{\partial b_i} = A_L \text{diag}(\sigma'(\tilde{y}_{L-1})) A_{L-1} \dots \text{diag}(\sigma'(\tilde{y}_{i+1})) A_{i+1} \text{diag}(\sigma'(\tilde{y}_i))$

또한 $\sigma'(z) = \left(\frac{1}{1+e^{-z}}\right)' = \frac{e^{-z}}{(1+e^{-z})^2} \leq \frac{1}{4}$ 이므로 앞의 $\text{diag}(\sigma'(\tilde{y}_j))$ 는 충분히 작지 않고,

또한 $A_1, \dots, A_{\ell}$ 이 not too large 이므로 $A_L, A_{L-1}, \dots, A_{i+1}$ 중 small한 것이 적어도 $\frac{\partial y_L}{\partial b_i}$ 는 small

$\frac{\partial y_L}{\partial A_i} = \text{diag}(\sigma'(\tilde{y}_i)) \left(\frac{\partial y_L}{\partial y_i}\right)^T y_{i-1}^T = \text{diag}(\sigma'(\tilde{y}_i)) \left(\frac{\partial y_L}{\partial y_{L-1}} \dots \frac{\partial y_{L-1}}{\partial y_i}\right)^T y_{i-1}^T$

$= \text{diag}(\sigma'(\tilde{y}_i)) (A_L \text{diag}(\sigma'(\tilde{y}_{L-1})) A_{L-1} \dots \text{diag}(\sigma'(\tilde{y}_{i+1})) A_{i+1})^T y_{i-1}^T$

이때 앞의 $\text{diag}(\sigma'(\tilde{y}_i))$ 가 충분히 작지 않고 $A_L, \dots, A_{i+1}$ 중 small한 것이 적어도,

$i \in \{1, \dots, \ell\}$ 이므로 $y_{i-1} = \sigma(\tilde{y}_{i-1})$, $1 \leq i \leq \ell$ 이므로 $y_i^T = x^T$ 는 충분히 작지 않고

$\partial y_L / \partial A_i$ ($i=1, 2, \dots, \ell$) 는 small.

(2) $\frac{\partial y_L}{\partial b_i}$ 와 $\frac{\partial y_L}{\partial A_i}$ 는 not too large, $\tilde{y}_j$ ($j = \ell+1, \dots, L-1$) 이 큰 값일 때

$\sigma' \rightarrow 0$ 이므로 (componentwise) $\text{diag}(\sigma'(\tilde{y}_{L-1})), \dots, \text{diag}(\sigma'(\tilde{y}_{\ell+1}))$ 는 small $\Rightarrow \frac{\partial y_L}{\partial b_i}$ 는 small

또한 $\frac{\partial y_L}{\partial A_i}$ 는 $\text{diag}(\sigma'(\tilde{y}_{L-1})), \dots, \text{diag}(\sigma'(\tilde{y}_{\ell+1}))$ 는 small 하고 y_{i-1} component는 not too large 이므로 $\partial y_L / \partial A_i$ 는 small.

Problem 3: Two forms of momentum SGD. There are two forms for the (non-Nesterov) momentum SGD. Form I is

$$\theta^{k+1} = \theta^k - \alpha g^k + \beta(\theta^k - \theta^{k-1})$$

for $k = 0, 1, \dots$, where $\theta^{-1} = \theta^0$. This form is more commonly invoked in mathematical discussions as it makes the “momentum term” $\beta(\theta^k - \theta^{k-1})$ clearly visible. Form II is

$$\begin{aligned} v^{k+1} &= g^k + \beta v^k \\ \theta^{k+1} &= \theta^k - \alpha v^{k+1} \end{aligned}$$

for $k = 0, 1, \dots$, where $v^0 = 0$. This is the form implemented in PyTorch with the option `Nesterov=false`. Show that the two forms are equivalent in the sense that given a starting point $\theta^0 \in \mathbb{R}^n$ and a sequence of stochastic gradients $g^0, g^1, \dots \in \mathbb{R}^n$, Forms I and II produce the same $\theta^1, \theta^2, \dots$ sequence.

Hint. Use induction.

Form I : $\theta = \theta_1 \geq \theta_2$, Form II : $\theta = \theta_2 \geq \theta_1$,

$$\begin{aligned} k=1 & ; \quad \theta_1^1 = \theta_1^0 - \alpha g^0 + \beta(\theta_1^0 - \theta_1^{-1}) = \theta_1^0 - \alpha g^0 = \theta_1^0 - \alpha g^0 \\ & \quad \theta_2^1 = \theta_2^0 - \alpha v^1 = \theta_2^0 - \alpha(g^0 + \beta v^0) = \theta_2^0 - \alpha g^0 \end{aligned} \quad \theta_1^1 = \theta_2^1$$

$k \in \mathbb{Z}_n = \{1, 2, \dots, n\}$ 일 때 모든 k 에 대해 ; $\theta_1^k = \theta_2^k$, $\dots$, $\theta_1^n = \theta_2^n$

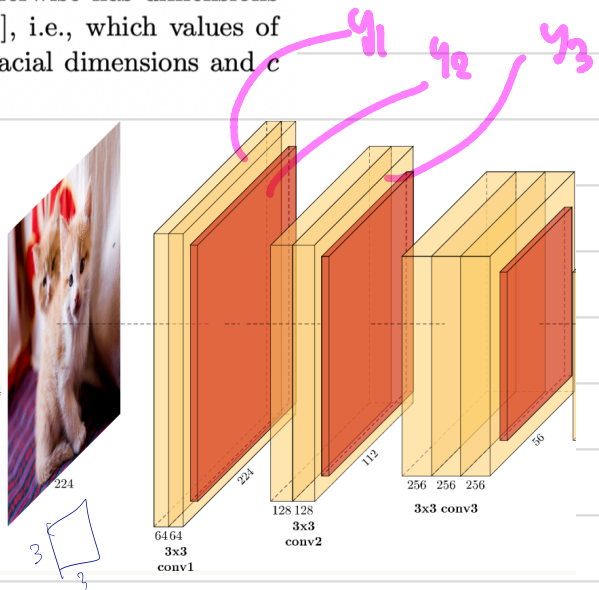
$$k=n+1 ; \quad \theta_1^{n+1} = \theta_1^n - \alpha g^n + \beta(\theta_1^n - \theta_1^{n-1})$$

$$\begin{aligned} \theta_2^{n+1} &= \theta_2^n - \alpha v^{n+1} = \theta_2^n - \alpha(g^n + \beta v^n) = \theta_2^n - \alpha g^n + \beta(-\alpha v^n) = \theta_2^n - \alpha g^n + \beta(\theta_2^n - \theta_2^{n-1}) \\ &= \theta_2^n - \alpha g^n + \beta(\theta_1^n - \theta_1^{n-1}) = \theta_1^{n+1} \end{aligned}$$

따라서 모든 $k \in \mathbb{Z}_n$ 에 대해 $\theta_1^k = \theta_2^k$ $\forall k \in \mathbb{N}$, 즉 Form I과 Form II는 같다. $\square$

Problem 4: The *receptive field* of a neuron is the set of input pixels the neuron value depends on. Consider the VGG16 network, and consider the intermediate values y_1 , y_2 , and y_3 , which are respectively the outputs of the second convolutional layer, the first maxpool layer, and the second maxpool layer. Assume the input X has a batch size 1 and otherwise has dimensions $3 \times 224 \times 224$. For $p = 1, 2, 3$, describe the receptive field of $y_p[k, i, j]$, i.e., which values of $X[c, m, n]$ does $y_p[k, i, j]$ depend on? Here, m, n and i, j denote the spacial dimensions and c and k denote the channels.

```
class VGG16(nn.Module) :
    def __init__(self) :
        super(VGG16, self).__init__()
        self.conv_layer1 = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1),      #64x224x224
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),      #64x224x224
            nn.ReLU() )
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2) #64x112x112
        self.conv_layer2 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding=1),    #128x112x112
            nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),    #128x112x112
            nn.ReLU() )
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2) #128x56x56
```



$$y_1[k, i, j] \leftarrow \text{1st conv. layer} [k_0, i_0 + \lambda_0, j_0 + j_0] \quad (\lambda_0 = -1 \sim 1, j_0 = -1 \sim 1, k_0 = 1 \sim 64)$$

$$\leftarrow X[k_0, i_0 + \lambda_0, j_0 + j_0] \quad (\lambda_0 = -2 \sim 2, j_0 = -2 \sim 2, k_0 = 1 \sim 3)$$

$$y_2[k, i, j] \leftarrow y_1[k_0, 2i_0 + \lambda_0, 2j_0 + j_0] \quad (\lambda_0 = -1, j_0 = -1, k_0 = 1 \sim 64)$$

$$\leftarrow X[k_0, 2i_0 + \lambda_0, 2j_0 + j_0] \quad (\lambda_0 = -3 \sim 3, j_0 = -3 \sim 3, k_0 = 1 \sim 3)$$

$$y_3[k, i, j] \leftarrow \text{2nd conv. layer} [k_0, 2i_0 + \lambda_0, 2j_0 + j_0] \quad (\lambda_0 = -1, j_0 = -1, k_0 = 1 \sim 128)$$

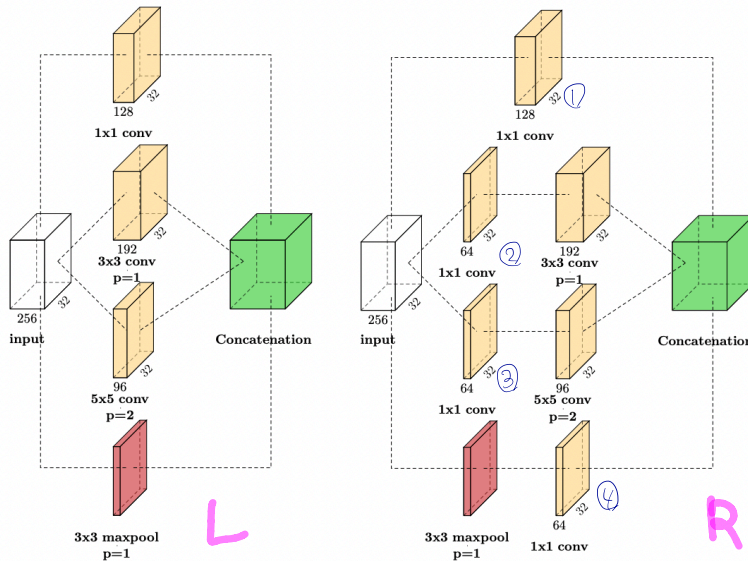
$$\leftarrow \text{maxpool} [k_0, 2i_0 + \lambda_0, 2j_0 + j_0] \quad (\lambda_0 = -3 \sim 3, j_0 = -3 \sim 3, k_0 = 1 \sim 128)$$

$$\leftarrow y_1[k_0, 4i_0 + \lambda_0, 4j_0 + j_0] \quad (\lambda_0 = -7 \sim 7, j_0 = -7 \sim 7, k_0 = 1 \sim 64)$$

$$\leftarrow X[k_0, 4i_0 + \lambda_0, 4j_0 + j_0] \quad (\lambda_0 = -9 \sim 9, j_0 = -9 \sim 9, k_0 = 1 \sim 3)$$

(224x224x3의 입력을 받아 ~224x224x3의 출력으로 변환.)

Problem 5: Consider the naïve inception module and the inception module with 1×1 “bottleneck” convolutions. Assume the input has 32×32 spatial dimensions and 256 channels. The numbers of output channels of each convolution operation are specified in the figure. Assume all convolutions use biases. A nonlinear activation function is applied after every convolution.



$\Rightarrow$ R은 L과 달리 channel이 많음

처음에 축소시킨 후 다시 convolution한다.

Between the two modules, compare:

(i) the number of trainable parameters and

(L) 1×1 conv : $(256 \times 1^2 + 1) \times 128 = 32896$

3×3 conv : $(256 \times 3^2 + 1) \times 192 = 442560$

5×5 conv : $(256 \times 5^2 + 1) \times 96 = 614496$

1089552 개

$\approx \times 1/3$

(R) 1×1 conv ① : $(256 \times 1^2 + 1) \times 128 = 32896$

1×1 conv ② : $(256 \times 1^2 + 1) \times 64 = 16448$

1×1 conv ③ : $(256 \times 1^2 + 1) \times 64 = 16448$

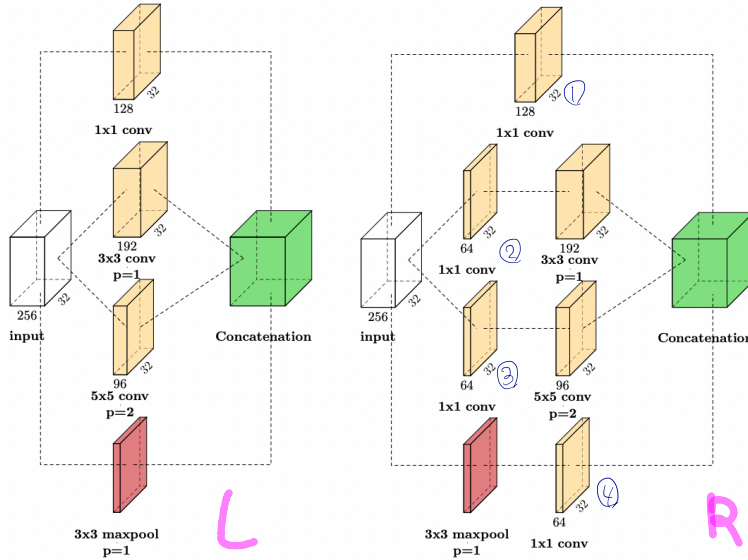
3×3 conv : $(64 \times 3^2 + 1) \times 192 = 110784$

5×5 conv : $(64 \times 5^2 + 1) \times 96 = 153696$

1×1 conv ④ : $(256 \times 1^2 + 1) \times 64 = 16448$

346720 개

Problem 5: Consider the naïve inception module and the inception module with 1×1 “bottleneck” convolutions. Assume the input has 32×32 spatial dimensions and 256 channels. The numbers of output channels of each convolution operation are specified in the figure. Assume all convolutions use biases. A nonlinear activation function is applied after every convolution.



Between the two modules, compare:

- (ii) the number of additions, multiplications, and activation function evaluations (separately for the three types of operations) required to forward-evaluate the module.

(additions)

$$(L) \quad 1 \times 1 \text{ conv} : 1^2 \times 256 \times 128 \times 32^2 = 33554432$$

$$3 \times 3 \text{ conv} : 3^2 \times 256 \times 192 \times 32^2 = 452984832$$

$$5 \times 5 \text{ conv} : 5^2 \times 256 \times 96 \times 32^2 = 629145600$$

$$11156486474$$

$$(R) \quad 1 \times 1 \text{ conv } ① : 1^2 \times 256 \times 128 \times 32^2 = 33554432$$

$$1 \times 1 \text{ conv } ② : 1^2 \times 256 \times 64 \times 32^2 = 1677216$$

$$1 \times 1 \text{ conv } ③ : 1^2 \times 256 \times 64 \times 32^2 = 1677216$$

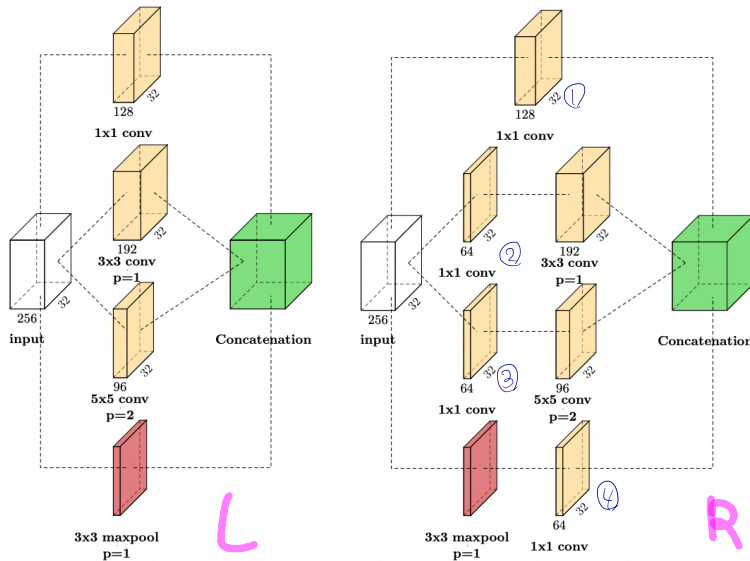
$$3 \times 3 \text{ conv} : 3^2 \times 64 \times 192 \times 32^2 = 113246208$$

$$5 \times 5 \text{ conv} : 5^2 \times 64 \times 96 \times 32^2 = 15788400$$

$$1 \times 1 \text{ conv } ④ : 1^2 \times 256 \times 64 \times 32^2 = 1677216$$

$$\Rightarrow 3354186874$$

Problem 5: Consider the naïve inception module and the inception module with 1×1 “bottleneck” convolutions. Assume the input has 32×32 spatial dimensions and 256 channels. The numbers of output channels of each convolution operation are specified in the figure. Assume all convolutions use biases. A nonlinear activation function is applied after every convolution.



(in fact, the number of multiplications and the number of additions are same.)

Between the two modules, compare:

- (ii) the number of additions, multiplications, and activation function evaluations (separately for the three types of operations) required to forward-evaluate the module.

(multiplications)

$$(L) \quad 1 \times 1 \text{ conv} : 1^2 \times 256 \times 128 \times 32^2 = 33554432$$

$$3 \times 3 \text{ conv} : 3^2 \times 256 \times 192 \times 32^2 = 452984832$$

$$5 \times 5 \text{ conv} : 5^2 \times 256 \times 96 \times 32^2 = 629145600$$

$$1115648640$$

$$(R) \quad 1 \times 1 \text{ conv } ① : 1^2 \times 256 \times 128 \times 32^2 = 33554432$$

$$1 \times 1 \text{ conv } ② : 1^2 \times 256 \times 64 \times 32^2 = 1677216$$

$$1 \times 1 \text{ conv } ③ : 1^2 \times 256 \times 64 \times 32^2 = 1677216$$

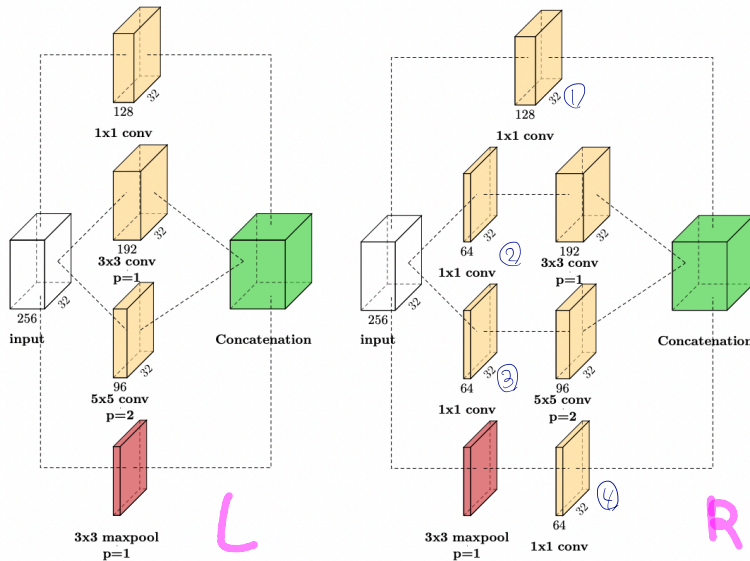
$$3 \times 3 \text{ conv} : 3^2 \times 64 \times 192 \times 32^2 = 113246208$$

$$5 \times 5 \text{ conv} : 5^2 \times 64 \times 96 \times 32^2 = 15786400$$

$$1 \times 1 \text{ conv } ④ : 1^2 \times 256 \times 64 \times 32^2 = 1677216$$

$$\Rightarrow 3354186880$$

Problem 5: Consider the naïve inception module and the inception module with 1×1 “bottleneck” convolutions. Assume the input has 32×32 spatial dimensions and 256 channels. The numbers of output channels of each convolution operation are specified in the figure. Assume all convolutions use biases. A nonlinear activation function is applied after every convolution.



Between the two modules, compare:

- (ii) the number of additions, multiplications, and activation function evaluations (separately for the three types of operations) required to forward-evaluate the module.

(Activation function evaluations)

$$\begin{aligned}
 \text{L)} \quad & 1 \times 1 \text{ conv} : 128 \times 32^2 = 131072 \\
 & 3 \times 3 \text{ conv} : 192 \times 32^2 = 196608 \\
 & 5 \times 5 \text{ conv} : 96 \times 32^2 = 98304
 \end{aligned}
 \quad \left. \vphantom{\begin{aligned} & 1 \times 1 \text{ conv} : 128 \times 32^2 = 131072 \\ & 3 \times 3 \text{ conv} : 192 \times 32^2 = 196608 \\ & 5 \times 5 \text{ conv} : 96 \times 32^2 = 98304 \end{aligned}} \right\} 425984$$

$$\begin{aligned}
 \text{R)} \quad & 1 \times 1 \text{ conv } ① : 128 \times 32^2 = 131072 \\
 & 1 \times 1 \text{ conv } ② : 64 \times 32^2 = 65536 \\
 & 1 \times 1 \text{ conv } ③ : 64 \times 32^2 = 65536 \\
 & 3 \times 3 \text{ conv} : 192 \times 32^2 = 196608 \\
 & 5 \times 5 \text{ conv} : 96 \times 32^2 = 98304 \\
 & 1 \times 1 \text{ conv } ④ : 64 \times 32^2 = 65536
 \end{aligned}
 \quad \left. \vphantom{\begin{aligned} & 1 \times 1 \text{ conv } ① : 128 \times 32^2 = 131072 \\ & 1 \times 1 \text{ conv } ② : 64 \times 32^2 = 65536 \\ & 1 \times 1 \text{ conv } ③ : 64 \times 32^2 = 65536 \\ & 3 \times 3 \text{ conv} : 192 \times 32^2 = 196608 \\ & 5 \times 5 \text{ conv} : 96 \times 32^2 = 98304 \\ & 1 \times 1 \text{ conv } ④ : 64 \times 32^2 = 65536 \end{aligned}} \right\} 622592$$